

Chapter 03

The Model Component

An end-to-end, hands-on tour of every model type LangChain can talk to — LLMs, Chat Models, closed-source APIs, open-source models, and embedding models — with working code for each.

WHAT THIS CHAPTER COVERS

- ▶ LLMs vs. Chat Models: the real difference
- ▶ Working with OpenAI, Anthropic Claude & Gemini
- ▶ Temperature & max tokens explained
- ▶ Open-source models: cost, control, trade-offs
- ▶ Using Hugging Face Inference API & local models
- ▶ OpenAI & Hugging Face embedding models
- ▶ Building a document similarity search app
- ▶ Cosine similarity for semantic retrieval

The Model Component

RECAP — CHAPTER 2

Chapter 2 introduced the six core building blocks of LangChain — Models, Prompts, Chains, Indexes, Memory, and Agents — and explained that Models are the interface through which every other component talks to an AI model. This chapter goes fully hands-on with that first component: every model type LangChain supports, and how to write working code against each one.

1. The Model Component: Definition & Two Types

Definition

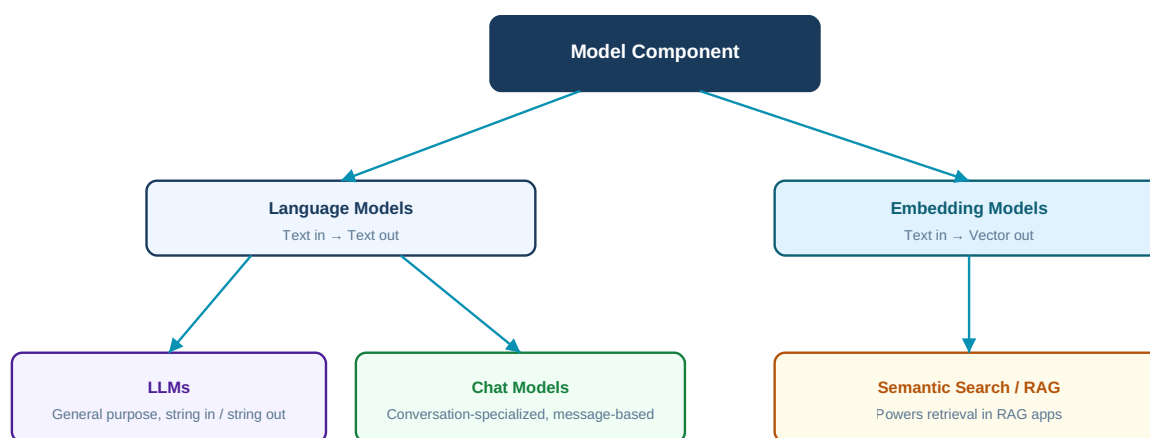
The **Model Component** in LangChain is the part of the framework designed to facilitate interaction with various language models and embedding models. In simple terms, many different AI models exist in the world, and each provider's API behaves differently. The Model Component gives you a single, *common interface* so you can connect to almost any AI model without rewriting your integration code for each provider.

Why It Exists

Without a standard interface, switching from one provider's model to another (say, from OpenAI to Anthropic) would mean rewriting large parts of an application's code, since every company structures its API calls differently. LangChain's Model Component abstracts this away: the same `.invoke()` pattern works regardless of which underlying provider is powering the model.

The Two Types of Models

Type	Input	Output	Used For
Language Models	Text (a prompt)	Text (a reply)	Chatbots, text generation, summarization, Q&A, code generation
Embedding Models	Text	A vector (series of numbers)	Semantic search, RAG-based applications



Every model in LangChain is accessed through this one common interface.

Figure 3.1 — The Model Component splits into Language Models (further split into LLMs and Chat Models) and Embedding Models.

Embeddings are vectors that represent the contextual meaning of a piece of text — two pieces of text with similar meaning end up with vectors that are close together in vector space. This is what makes semantic search possible.

IMPORTANT

The Model Component is not a model itself — it is an interface. It standardizes how you call any language model or embedding model, regardless of which company built it.

🔄 Quick Revision — The Model Component

- The Model Component is LangChain's common interface for interacting with AI models.
- There are exactly two categories: Language Models and Embedding Models.
- Language Models take text in and return text out.
- Embedding Models take text in and return a vector (a series of numbers) out.
- Language Models power chatbot-style applications; Embedding Models power semantic search and RAG.

2. Language Models: LLMs vs. Chat Models

Definition

Language models take a text input, process it, and return a text output. Within this category, LangChain distinguishes between two sub-types: *LLMs* and *Chat Models*.

1. LLMs

LLMs are **general-purpose** models: they take a plain string as input and return a plain string as output. They can be used for almost any NLP task — text generation, summarization, code generation, question answering — but they were not specifically designed for multi-turn conversation.

2. CHAT MODELS

Chat Models are language models **specialized for conversational tasks**. They take a sequence of messages as input and return chat messages as output, meaning they can understand an entire conversation (not just a single prompt) and reply in a structured, message-based format.

How They Differ

Aspect	LLMs	Chat Models
Purpose	Free-form text generation: Q&A, summarization, translation	Multi-turn conversation between user and AI
Training	Trained generally on large text corpora (books, articles, Wikipedia)	Trained the same way, then additionally fine-tuned on chat/conversation datasets
Memory	No concept of memory — each call is independent	Support conversation history within a session
Role Awareness	No roles — cannot be assigned a persona via a system message	Understands distinct roles: system, user, and assistant
Example Models	Older completion-style models (e.g. GPT-3.5-Turbo-Instruct)	GPT-4, Claude, Gemini
Best For	Text generation, summarization, translation, code generation	Chatbots, virtual assistants, customer support bots, AI tutors

REAL WORLD INSIGHT

Most Generative AI applications built today are conversational, which is why the industry — and LangChain itself — is steadily shifting its focus and support toward Chat Models. LLM support in newer LangChain versions is being phased out; new projects are recommended to use Chat Models instead.

INTERVIEW TIP

If asked to differentiate LLMs from Chat Models, lead with the input/output shape: LLMs take and return plain strings, while Chat Models take and return a sequence of role-tagged messages. Everything else (memory, role awareness, use case) follows from that core distinction.

Quick Revision — LLMs vs. Chat Models

- LLMs are general-purpose: string in, string out.
- Chat Models are conversation-specialized: sequence of messages in, chat message out.
- Only Chat Models support conversation history and role assignment (system/user/assistant).
- LangChain's ecosystem is steadily shifting away from LLMs toward Chat Models.
- Use LLMs for one-off generation tasks; use Chat Models for anything conversational.

3. Setting Up the Development Environment

How It Works

Before writing any model-interaction code, a consistent project setup is created so every example in this chapter runs smoothly.

1. CREATE AND ACTIVATE A VIRTUAL ENVIRONMENT

```
# create the virtual environment
python -m venv venv

# activate it (Windows)
venv\Scripts\activate
```

2. INSTALL THE REQUIRED LIBRARIES

All required package names are listed in a `requirements.txt` file and installed in one shot:

```
pip install -r requirements.txt
```

3. STORE API KEYS IN A `.ENV` FILE

Secret keys should never be hard-coded directly into source files. Instead, they are placed in a `.env` file at the project root and loaded at runtime via the `python-dotenv` package:

```
# .env
OPENAI_API_KEY="sk-..."
ANTHROPIC_API_KEY="sk-ant-..."
GOOGLE_API_KEY="AIza..."
HUGGINGFACEHUB_ACCESS_TOKEN="hf_..."
```

4. VERIFY THE INSTALLATION

```
import langchain
print(langchain.__version__)
```

BEST PRACTICE

Keep each provider's API key under the exact variable name LangChain expects (e.g. `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`). The `load_dotenv()` function locates keys by name — renaming a variable will make it invisible to the library that needs it.

Quick Revision — Environment Setup

- Create an isolated virtual environment per project with `python -m venv venv`.
- Install all dependencies at once from a `requirements.txt` file.
- Store all API keys in a `.env` file, never directly in code.
- Load environment variables at runtime with `load_dotenv()` from `python-dotenv`.
- Each provider's key must use its exact expected variable name.

4. Working with Closed-Source Models: OpenAI

Definition

Closed-source (proprietary) models are paid models hosted entirely on a provider's own servers, accessed only through that provider's API. OpenAI's GPT family is the most widely used example.

Prerequisite

Using OpenAI's models requires an API key from `platform.openai.com`, which in turn requires the account to hold a minimum of \$5 in credit — OpenAI no longer provides free credits to new users. The key is generated from **Settings** → **API Keys** → **Create Secret Key** and stored in `.env` as `OPENAI_API_KEY`.

Example: LLM vs. Chat Model Code

The two code paths below show how minimal the difference is — only the imported class and the model name change.

LLM INTERFACE (GPT-3.5-TURBO-INSTRUCT)

```
from langchain_openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

llm = OpenAI(model="gpt-3.5-turbo-instruct")

# .invoke() sends a plain string, gets a plain
string back
result = llm.invoke("What is the capital of
India?")

print(result)
# Output: "The capital of India is New Delhi"
```

CHAT MODEL INTERFACE (GPT-4)

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model="gpt-4")

# .invoke() sends a message, gets a structured
AIMessage back
result = model.invoke("What is the capital of
India?")

print(result.content)
# Output: "New Delhi is the capital of India"
```

The `.invoke()` method is central to LangChain: Models, Chains, and Prompts all expose this same method, and calling it sends the request to the underlying model and returns its response.

Notice that the Chat Model's raw `result` object is not a plain string — it also carries metadata such as token counts for the prompt and the completion. Only `result.content` contains the actual answer text.

COMMON MISTAKE

Printing the raw `result` object from a Chat Model call and expecting clean text output. Chat Models return a message object with metadata attached — always access `result.content` to get just the answer.

Quick Revision — OpenAI Models in LangChain

- OpenAI's API requires a paid account with at least \$5 in credit.
- LLMs are accessed via the OpenAI class; Chat Models via ChatOpenAI.
- Both expose the same `.invoke()` method for sending a prompt and getting a response.
- LLM responses are plain strings; Chat Model responses are message objects — use `.content` to extract the text.
- LangChain's OpenAI integration lives in the `langchain_openai` package.

5. Key Inference Parameters: Temperature & Max Tokens

Temperature

DEFINITION

Temperature is a parameter that controls the randomness of a language model's output. It typically ranges from 0 to around 2, and directly affects how creative versus deterministic the responses are.

HOW IT WORKS

Temperature Range	Behavior	Good For
0 – 0.3	Deterministic, predictable	Factual answers, math, code generation
0.5 – 0.7	Balanced	General Q&A, explanations
0.9 – 1.2	Creative, varied	Story writing, jokes, marketing copy
1.5+	Highly random, very diverse	Brainstorming, ideation

```
model = ChatOpenAI(model="gpt-4", temperature=0)
```

Max Completion Tokens

DEFINITION

Max tokens (or `max_completion_tokens`) sets an upper limit on how many tokens the model's response can contain. A token can loosely be thought of as roughly one word (though tokenization is more nuanced than that in practice).

WHY IT EXISTS

Paid language model APIs charge **per token**, based on published per-million-token pricing. Restricting the maximum output length directly controls cost.

```
model = ChatOpenAI(model="gpt-4", max_completion_tokens=10)
# the response will be truncated to a maximum of 10 tokens
```

BEST PRACTICE

Set temperature close to 0 for deterministic tasks like code or math, and closer to 1.5 for creative writing tasks. Combine this with a sensible `max_completion_tokens` limit to keep API costs predictable.

Quick Revision — Temperature & Max Tokens

- Temperature controls randomness/creativity; range is roughly 0 to 2.
- Low temperature (0–0.3) = deterministic and factual; high temperature (1.5+) = highly creative.
- Max tokens caps the length of the model's response.
- Paid APIs bill per token, so max tokens directly controls cost.
- A token is roughly (but not exactly) equal to one word.

6. Other Closed-Source Providers: Anthropic & Google Gemini

Why It Matters

LangChain's biggest practical advantage is consistency: switching from one provider to another requires changing only the import statement, the class name, and the model name — the rest of the code (`.invoke()`, `.content`) stays identical.

Anthropic Claude

Claude is developed by Anthropic and, on several benchmarks, is reported to outperform GPT models — making it a common choice in production systems. Like OpenAI, it is a paid API requiring an account and API key from `console.anthropic.com`, stored as `ANTHROPIC_API_KEY`.

Google Gemini

Gemini is Google's family of models, accessed with an API key created through Google's Gemini API documentation site and stored as `GOOGLE_API_KEY`.

ANTHROPIC CLAUDE

```
from langchain_anthropic import ChatAnthropic
from dotenv import load_dotenv

load_dotenv()

model = ChatAnthropic(model="claude-3-5-sonnet-20241022")

result = model.invoke("What is the capital of India?")
print(result.content)
```

GOOGLE GEMINI

```
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv

load_dotenv()

model = ChatGoogleGenerativeAI(model="gemini-1.5-pro")

result = model.invoke("What is the capital of India?")
print(result.content)
```

Aside from the import path, class name, and model string, the code structure for OpenAI, Anthropic, and Gemini is essentially identical — this is the direct payoff of LangChain's standardized Model Component.

IMPORTANT

Each provider requires its API key to be stored under its exact expected environment-variable name (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GOOGLE_API_KEY`). `load_dotenv()` cannot rename or guess these for you.

Quick Revision — Anthropic & Gemini

- ChatAnthropic connects to Claude models; ChatGoogleGenerativeAI connects to Gemini models.
- Both are paid, closed-source APIs requiring their own account and API key.
- Claude is widely reported to match or beat GPT models on several benchmarks.
- Switching providers in LangChain changes only the import, class, and model name.
- The `.invoke()` and `.content` pattern remains identical across all three providers.

7. Open Source Models

Definition

Open source models are freely available AI models that can be downloaded, modified, fine-tuned, and deployed without restrictions from a central provider — unlike closed-source models such as GPT, Claude, and Gemini, which allow no control or customization beyond what the provider's API exposes.

Problem It Solves

Closed-source models have two structural drawbacks: the API costs money to use, and because the model lives on someone else's server, the user has zero control over it. Open source models solve both problems — once downloaded, there is no API cost and full freedom to fine-tune, modify, or deploy the model however needed.

Open Source vs. Closed Source

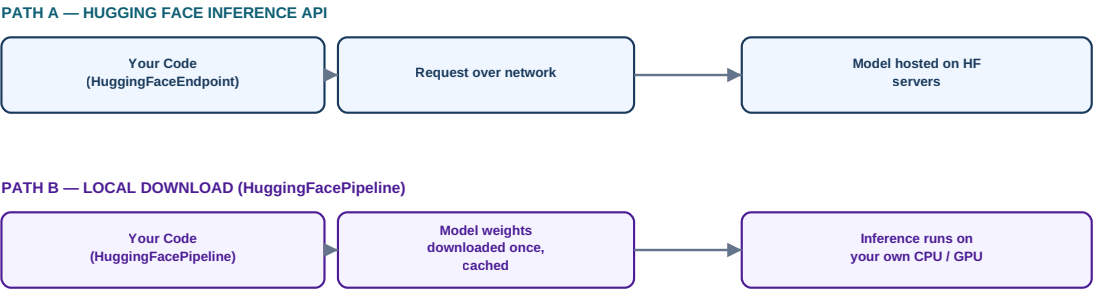
Aspect	Open Source	Closed Source
Cost	Free to run locally, no API charges	Pay per API call / token
Control	Full control — fine-tune, modify, deploy anywhere	Only the provider's infrastructure; zero control
Data Privacy	Data stays on your own machine	Data is sent to the provider's servers
Customization	Fully fine-tunable on your own datasets	Limited or no fine-tuning options
Deployment	Deploy on your own servers or cloud	No deployment option — API access only

Popular open source language models include **LLaMA** (Meta), **Mistral**, **Falcon**, and domain-specific models like **BLOOM**.

Hugging Face: The Home of Open Source Models

Hugging Face is the largest repository of open source models, hosting thousands of models across text generation, computer vision, and multimodal tasks. It is the standard place to search for and download an open source model.

Two Ways to Use Open Source Models



Path A needs no local hardware but has usage limits. Path B needs strong hardware but runs fully offline.

Figure 3.2 — Two access paths for open source models: the Hugging Face Inference API (no local hardware needed) versus a local download (runs fully offline).

Approach	How	Trade-off
Hugging Face Inference API	Call a hosted model over the network, like a closed-source API	Free tier has usage limits; paid beyond that; access to thousands of models
Local Download	Download model weights to your own machine and run inference locally	No usage limits, but needs strong hardware (RAM, GPU)

Disadvantages of Open Source Models

- **Hardware requirements:** running larger models locally requires expensive GPUs most individuals do not have.
- **Setup complexity:** preparing the local environment is more involved than simply calling a hosted API.
- **Less refinement:** open source models typically receive less Reinforcement Learning from Human Feedback (RLHF) fine-tuning, so responses can feel less polished than closed-source equivalents.
- **Limited multimodal support:** most freely available open source models are text-only; multimodal (image/audio) open source options are comparatively rare.

COMMON MISTAKE

Attempting to run a multi-billion-parameter open source model locally on a machine with limited RAM and no dedicated GPU. This can make the system unresponsive for many minutes and, on constrained hardware, may require a restart. Match model size to available hardware.

🔄 Quick Revision — Open Source Models

- Open source models can be freely downloaded, fine-tuned, and deployed without a central provider.
- They remove API cost and give full control, at the expense of needing local hardware.
- Hugging Face is the largest repository for finding open source models.
- Two usage paths exist: the Hugging Face Inference API, or downloading and running locally.
- Trade-offs vs. closed source: lower cost and more control, but weaker hardware performance and less RLHF refinement.

8. Using Open Source Models in LangChain

Example Model: TinyLlama

Both approaches below use **TinyLlama-1.1B-Chat**, a small (1.1 billion parameter) fine-tuned version of LLaMA — chosen because it is light enough to download and run on modest hardware.

Approach 1: Hugging Face Inference API

This calls the model on Hugging Face's own servers — no local download happens. It requires a Hugging Face access token (**Settings** → **Access Tokens** → **Create New Token**) stored as `HUGGINGFACEHUB_ACCESS_TOKEN`.

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from dotenv import load_dotenv

load_dotenv()

# first configure the underlying LLM endpoint
llm = HuggingFaceEndpoint(
    repo_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation"
)

# then wrap it in the Chat interface
model = ChatHuggingFace(llm=llm)

result = model.invoke("What is the capital of India?")
print(result.content)

```

Approach 2: Local Download

This downloads the model weights, tokenizer, and configuration files (roughly 300–500 MB for TinyLlama) to the local machine on first run, then loads them into memory and runs inference locally — no API call at all.

```

from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline
import os

# optional: redirect the local model cache to a drive with more space
os.environ["HF_HOME"] = "D:/huggingface_cache"

llm = HuggingFacePipeline.from_model_id(
    model_id="TinyLlama/TinyLlama-1.1B-Chat-v1.0",
    task="text-generation",
    pipeline_kwargs=dict(
        temperature=0.5,
        max_new_tokens=100
    )
)

model = ChatHuggingFace(llm=llm)

result = model.invoke("What is the capital of India?")
print(result.content)

```

The exact same repo ID identifies the model in both approaches — only the class (`HuggingFaceEndpoint` vs. `HuggingFacePipeline`) and where the computation happens differ.

BEST PRACTICE

On machines with limited disk space on the default drive, set the `HF_HOME` environment variable before loading the model to redirect the download cache to a drive with more free space.

Quick Revision — Running Open Source Models

- `HuggingFaceEndpoint` calls a model hosted on Hugging Face's servers (API-based, no download).
- `HuggingFacePipeline` with `from_model_id` downloads and runs the model locally.
- Both are wrapped in `ChatHuggingFace` to get the same `.invoke()` / `.content` interface.
- Local inference speed depends heavily on available RAM and whether a GPU is present.
- The `HF_HOME` environment variable can redirect where downloaded model files are cached.

9. Embedding Models

Definition

Embedding models convert text into a vector — a fixed-length series of numbers that captures the contextual meaning of that text. Texts with similar meaning produce vectors that sit close together in vector space.

Two Core Methods

Method	Purpose	Input	Output
<code>embed_query()</code>	Embed a single piece of text (e.g. a user's question)	One string	One vector
<code>embed_documents()</code>	Embed multiple pieces of text at once (e.g. a document set)	A list of strings	A list of vectors (a 2D list)

Example: OpenAI Embeddings

```
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()

embedding = OpenAIEmbeddings(model="text-embedding-3-large", dimensions=32)

# single query
result = embedding.embed_query("Delhi is the capital of India")
print(str(result))

# multiple documents at once
docs = [
    "Delhi is the capital of India",
    "Kolkata is the capital of West Bengal",
    "Paris is the capital of France"
]
doc_results = embedding.embed_documents(docs)
print(str(doc_results)) # a 2D list: one vector per document
```

Choosing a Vector Dimension

OpenAI embedding models let you request a specific output dimension (e.g. 32, 300, 1536). The default length is **1536** for the small model and **3072** for the large model.

Vector Size	Context Captured	Cost
Smaller (e.g. 32–300)	Less contextual detail	Lower
Larger (e.g. 1536–3072)	More contextual detail	Higher

Embedding is comparatively cheap: OpenAI's pricing is roughly \$0.13 per million tokens processed, since the model only needs to output numbers rather than generated text.

Example: Open Source Embeddings (Local)

A popular free alternative is **sentence-transformers/all-MiniLM-L6-v2**, a small (~90 MB) sentence-transformer model that maps sentences and paragraphs to a 384-dimensional vector space, well suited for clustering and semantic search.

```
from langchain_huggingface import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

text = "Delhi is the capital of India"
result = embedding.embed_query(text)
print(str(result)) # a 384-dimension vector
```

REAL WORLD INSIGHT

OpenAI's embeddings are inexpensive and generally produce richer, more accurate context than most free alternatives. Open source embedding models are a solid free option, but in practice tend to be somewhat less accurate for semantic search tasks.

Quick Revision — Embedding Models

- `embed_query()` embeds a single string; `embed_documents()` embeds a list of strings at once.
- OpenAI embedding models let you choose the output vector dimension (default 1536 / 3072).
- Larger vectors capture more context but cost more to generate.
- `sentence-transformers/all-MiniLM-L6-v2` is a small, free, local embedding model producing 384-dimension vectors.
- Embedding is much cheaper per token than text generation, since the output is just numbers.

10. Building a Document Similarity Search Application

Definition

A minimal application that, given a set of candidate documents and a user query, identifies which document is most semantically related to that query — the same core mechanism that powers retrieval in RAG-based applications.

How It Works

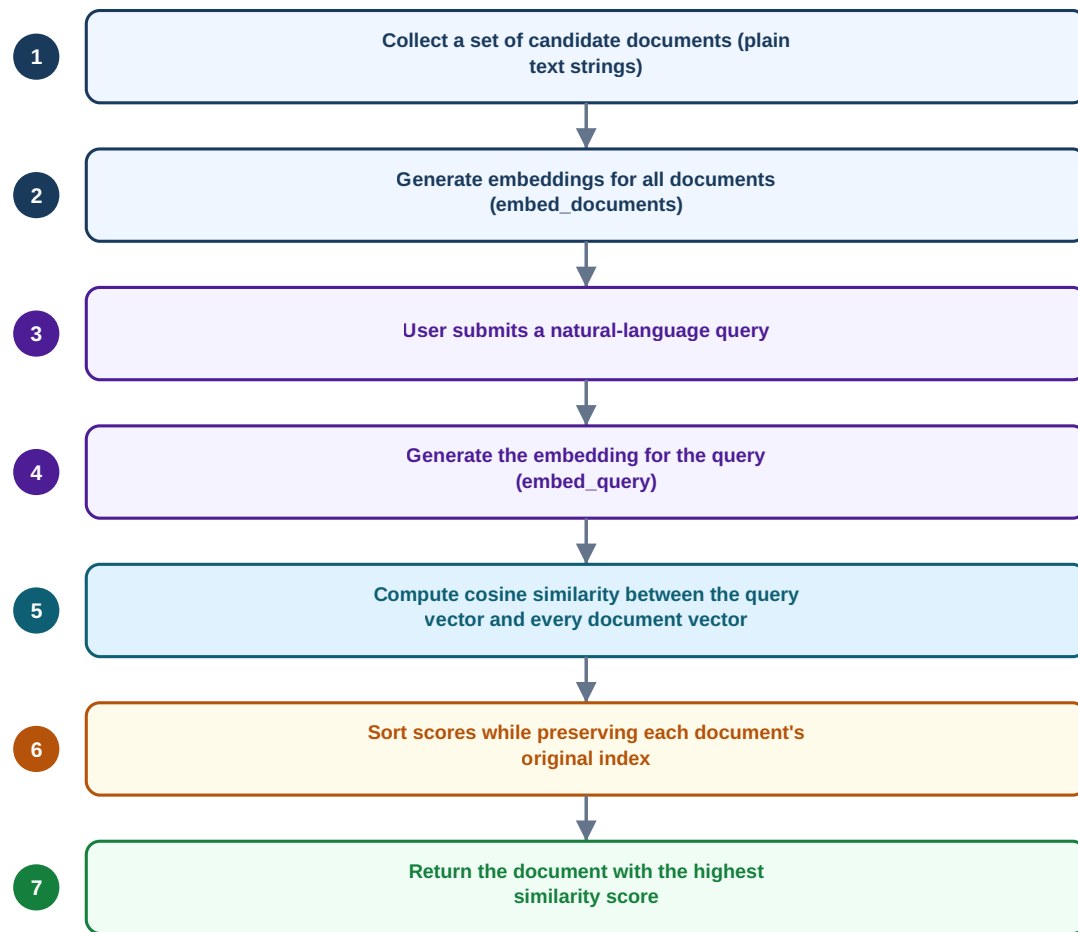


Figure 3.3 — End-to-end flow of a document similarity search application.

Once both the document vectors and the query vector exist in the same vector space, **cosine similarity** (essentially the angle between two vectors) is used to measure how close the query vector is to each document vector. The document with the highest similarity score is the best match.

Example

```
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

load_dotenv()

embedding = OpenAIEmbeddings(model="text-embedding-3-large", dimensions=300)

# five candidate documents, each about a cricketer
documents = [
    "Virat Kohli is an Indian cricketer known for his aggressive batting...",
    "Jasprit Bumrah is an Indian fast bowler known for his unique action...",
    "...", # three more documents
]

query = "Tell me about Virat Kohli"

# step 1: embed all documents at once
document_embedding = embedding.embed_documents(documents)

# step 2: embed the query
query_embedding = embedding.embed_query(query)

# step 3: cosine similarity needs 2D lists for both arguments
scores = cosine_similarity([query_embedding], document_embedding)[0]

# step 4: pair each score with its original index before sorting,
# so the correct document can still be located afterward
index, score = sorted(list(enumerate(scores)), key=lambda x: x[1])[-1]

print(query)
print(documents[index])
print("Similarity score is:", score)
```

Why enumerate() Is Used Before Sorting

Sorting the similarity scores directly would lose track of which score belonged to which document, since sorting rearranges the list order. Wrapping the scores in `enumerate()` first attaches each score to its original index, so after sorting by score, the correct document can still be retrieved using that preserved index.

REAL WORLD INSIGHT

Recomputing document embeddings on every run is a costly operation. In a real application, document embeddings are generated once and stored in a **vector database**. Only the incoming query is embedded on the fly each time — this lookup process is called **retrieval**, and is the foundation of RAG-based applications covered in later chapters.

Quick Revision — Document Similarity Search

- The application embeds a fixed set of documents once, then embeds each incoming query.
- Cosine similarity measures closeness between the query vector and every document vector.
- `cosine_similarity()` from scikit-learn requires both inputs as 2D lists.
- `enumerate()` preserves each score's original document index through the sort.
- In production, document embeddings are pre-computed and stored in a vector database rather than regenerated on every run.

K. Key Takeaways

- `BaseLLM` is LangChain's standardized interface for talking to any language model or embedding model.
- `BaseLLM` takes text in, returns text out; split into general-purpose LLMs and conversation-specialized Chat Models.
- `BaseLLM` LLMs use plain strings and have no memory or roles; Chat Models use message sequences and support conversation history and role assignment.
- `OpenAI`, `Anthropic`, and `Google` are paid APIs accessed with near-identical LangChain code — only the class and model name change.
- `OpenAI` controls creativity/randomness; `OpenAI` caps response length and controls cost.
- `LocalLLM` removes API cost and gives full control and customization, at the cost of needing local hardware and receiving less refinement.
- `HuggingFace` is the largest repository for open source models, usable either via its Inference API or by downloading models locally.
- `OpenAI` converts text into vectors that capture contextual meaning, using `embed_query()` for single texts and `embed_documents()` for many at once.
- `OpenAI` uses cosine similarity between a query vector and a set of document vectors to find the most relevant match — the foundation of RAG retrieval.

R. Revision Sheet

Model Component

LangChain's common interface for connecting to any language or embedding model, regardless of provider.

LLM

A general-purpose language model: plain string in, plain string out, no memory or roles.

Chat Model

A conversation-specialized language model: message sequence in, message out, with memory and role awareness.

Temperature

Parameter (0–2) controlling output randomness — low for deterministic tasks, high for creative tasks.

Max Tokens

Caps the length of a model's response; directly controls per-token API cost.

Open Source Model

A freely downloadable, fine-tunable, self-hostable model (e.g. LLaMA, Mistral) with no API cost but higher hardware demands.

Hugging Face

The largest repository of open source models, usable via its Inference API (hosted) or local download.

Embedding Model

Converts text into a numeric vector that captures its contextual meaning; used for semantic search.

`embed_query()` / `embed_documents()`

Embed a single string, or a list of strings at once, respectively.

Cosine Similarity

A measure of the angle between two vectors, used to score how semantically close a query is to a document.

I. Interview Questions

- Q1.** What problem does LangChain's Model Component solve when working with multiple AI providers?
- Q2.** What is the fundamental difference between an LLM and a Chat Model in LangChain?
- Q3.** Why is LangChain's ecosystem gradually shifting support away from LLMs and toward Chat Models?
- Q4.** What does the temperature parameter control, and how would you set it differently for a code-generation task versus a story-writing task?
- Q5.** Why do paid LLM APIs commonly expose a max tokens (or max_completion_tokens) parameter?
- Q6.** What are the key trade-offs between using a closed-source model API and an open source model?
- Q7.** What are the two ways to use an open source model from Hugging Face in LangChain, and how do they differ?
- Q8.** What is the difference between `embed_query()` and `embed_documents()`, and when would you use each?
- Q9.** Why must both arguments passed to scikit-learn's `cosine_similarity()` be 2D lists?
- Q10.** In a document similarity search implementation, why is `enumerate()` applied to the similarity scores before sorting them?

F. Further Reading

- [LangChain Python Documentation — Chat Models & LLMs](#)
- [OpenAI API Documentation — Models & Pricing](#)
- [Anthropic Claude Documentation — User Guides](#)
- [Google AI for Developers — Gemini API Documentation](#)
- [Hugging Face Model Hub — Text Generation Models](#)

- Hugging Face — [sentence-transformers/all-MiniLM-L6-v2 model card](#)
- scikit-learn Documentation — [sklearn.metrics.pairwise.cosine_similarity](#)